

Lista de Treino - Estruturas de Repetição *while*

Algoritmos e Linguagem de Programação · 12 questões nível prova

REGRAS DA LISTA — resolva usando somente o que já foi visto em aula:

- `#include <stdio.h>` — nenhuma outra biblioteca
- `int main()` e `return 0;`
- Tipos `int`, `float`, `double` e `char`
- `printf` e `scanf` com `%d`, `%f` e `%c`
- Operadores `+`, `-`, `*`, `/`, `%`
- Comparações `==`, `!=`, `>`, `<`, `>=`, `<=` e lógicos `&&`, `||`
- `if` / `else if` / `else` e `while`
- Incremento `++` e decremento `--`

NÃO use (ainda não foi dado): `for`, `do...while`, `switch`, `break`, `continue`, vetores, matrizes, funções criadas por você, `math.h` ou ponteiros.

Dica que salva prova: ao ler um `char` logo depois de ter lido um número, deixe um espaço antes do `%c`: `scanf(" %c", &resposta);`. Sem esse espaço o programa "pula" a leitura, porque sobra o *Enter* no buffer do teclado.

Bloco 1 — Validação de entrada e contadores

Questão 1. Faça um programa que leia a nota de um aluno e só aceite valores entre 0 e 10. Enquanto o valor digitado for inválido, exiba uma mensagem de erro e peça a nota novamente. Ao receber uma nota válida, mostre a nota aceita e **quantas tentativas inválidas** o usuário fez.

Exemplo de execução:

```
Digite a nota (0 a 10): 15
Nota invalida! Digite novamente (0 a 10): -2
Nota invalida! Digite novamente (0 a 10): 8.5
Nota aceita: 8.5
Tentativas invalidas: 2
```

Questão 2. Escreva um programa de login. O usuário cadastra uma senha numérica e depois precisa digitá-la, tendo no máximo **3 tentativas**. A cada erro, informe quantas tentativas ainda restam. Se acertar dentro do limite, escreva `Acesso liberado!`; se esgotar as três, escreva `Conta bloqueada!`.

Exemplo de execução:

```
Cadastre sua senha (numero inteiro): 1234
Digite sua senha: 1111
Senha incorreta! Voce ainda tem 2 tentativa(s).
Digite sua senha: 9999
Senha incorreta! Voce ainda tem 1 tentativa(s).
Digite sua senha: 1234
Acesso liberado!
```

Questão 3. Leia uma quantidade n de valores (validando que n seja maior que zero) e em seguida leia n números reais, um de cada vez. Ao final, informe quantos foram negativos, quantos positivos, quantos iguais a zero e a média de todos os valores lidos.

Exemplo de execução:

```
Quantos valores serao lidos? 5
Valor 1: -3
Valor 2: 0
Valor 3: 7.5
Valor 4: -1
Valor 5: 2
Negativos: 2
Positivos: 2
Zeros: 1
Media: 1.10
```

Bloco 2 — Números e dígitos

Questão 4. Leia um número inteiro positivo (validando a entrada) e imprima **todos os seus divisores**. Depois informe a quantidade de divisores encontrados e diga se o número é primo ou não. Lembre-se: um número é primo quando possui exatamente dois divisores.

Exemplo de execução:

```
Digite um numero inteiro positivo: 0
Numero invalido! Digite novamente: 12
Divisores de 12: 1 2 3 4 6 12
Quantidade de divisores: 6
0 numero 12 nao e primo.
```

Questão 5. Leia um número inteiro positivo (validando a entrada) e imprima seus dígitos **na ordem inversa**, sem usar vetores. Mostre também quantos dígitos o número tem e a soma deles. *Dica:* `n % 10` devolve o último dígito e `n / 10` remove esse dígito.

Exemplo de execução:

```
Digite um numero inteiro positivo: 9207
Invertido: 7029
Quantidade de digitos: 4
Soma dos digitos: 18
```

Questão 6. Escreva um programa que imprima todos os números entre 1000 e 2000 que sejam divisíveis por 7 e, ao mesmo tempo, **não** sejam divisíveis por 5. Ao final, mostre quantos números foram encontrados e a soma de todos eles. Este programa não lê nada do teclado.

Saída esperada (resumida):

```
1001 1008 1022 1029 1036 1043 1057 ... 1988
Quantidade: 114
Soma: 170569
```

Bloco 3 — Séries e acumuladores

Questão 7. Leia um número inteiro n entre 0 e 12 (validando a entrada) e calcule o fatorial de n , mostrando **todo o desenvolvimento** do cálculo. Lembre-se de que $0! = 1$.

Exemplo de execução:

```
Digite um número de 0 a 12: 20
Número inválido! Digite de 0 a 12: 5
5! = 5 x 4 x 3 x 2 x 1 = 120
```

Questão 8. Leia uma quantidade n de termos (validando $n > 0$) e imprima os n primeiros termos da sequência de Fibonacci, na qual cada termo é a soma dos dois anteriores, começando por 0 e 1. Atenção ao caso $n = 1$.

Exemplo de execução:

```
Quantos termos? 8
0 1 1 2 3 5 8 13
```

Questão 9. Faça um programa que leia números inteiros até que o usuário digite **0**. Ao encerrar, mostre quantos números foram lidos (sem contar o zero), o maior, o menor e a média deles. Se nenhum número for digitado antes do zero, escreva Nenhum valor foi digitado.

Exemplo de execução:

```
Digite um valor (0 encerra): 12
Digite um valor (0 encerra): -5
Digite um valor (0 encerra): 30
Digite um valor (0 encerra): 7
Digite um valor (0 encerra): 0
Quantidade de valores: 4
Maior: 30
Menor: -5
Média: 11.00
```

Bloco 4 — Desafios nível prova

Questão 10. Duas cidades A e B têm populações e taxas anuais de crescimento diferentes. Leia a população e a taxa de cada uma, validando que **todos** os valores sejam maiores que zero, e calcule em quantos anos a população de A alcança ou ultrapassa a de B. Trate os dois casos especiais: (a) A já é maior ou igual a B; (b) A nunca alcançará B porque sua taxa não é maior. Ao final, pergunte se o usuário deseja repetir a operação e continue enquanto ele responder S ou s.

Exemplo de execução:

```
Populacao da cidade A: 80000
Taxa anual de A em porcentagem: 3
Populacao da cidade B: 200000
Taxa anual de B em porcentagem: 1.5
Serao necessarios 63 anos.
Populacao final de A: 515033
Populacao final de B: 510964
Deseja repetir? (S/N): N
Programa encerrado.
```

Questão 11. Leia dois números inteiros de 1 a 10 indicando a primeira e a última tabuada a ser exibida (validando as entradas; a final não pode ser menor que a inicial). Imprima todas as tabuadas do intervalo, de 1 a 10, usando **um while dentro de outro while**.

Exemplo de execução:

```
Tabuada inicial (1 a 10): 0
Inválido! Tabuada inicial (1 a 10): 7
Tabuada final (7 a 10): 7
--- Tabuada do 7 ---
7 x 1 = 7
7 x 2 = 14
...
7 x 10 = 70
```

Questão 12. Simule um caixa eletrônico. O programa lê um valor inteiro de saque e informa a **menor quantidade possível** de notas de 100, 50, 20, 10, 5, 2 e 1 para compor esse valor, exibindo apenas as notas realmente utilizadas. O desconto de cada tipo de nota deve ser feito **com while**, subtraindo o valor da nota. Valores negativos exibem Valor inválido! e o programa se repete até o usuário digitar 0.

Exemplo de execução:

```
Valor do saque (0 encerra): 287
Notas para o saque de 287 reais:
2 nota(s) de 100
1 nota(s) de 50
1 nota(s) de 20
1 nota(s) de 10
1 nota(s) de 5
1 nota(s) de 2
Valor do saque (0 encerra): 0
Caixa encerrado.
```